P10

API RESTFUL CON PYTHON FLASK

CONTENIDOS

- Introducción a API RESTful
- Python Flask
- App básica en Flask
- Concepto de CRUD
- Métodos HTTP
- Códigos de estado
- Variables en URL
- Seguridad básica en Flask
- JSON Web Tokens

INTRODUCCIÓN: API RESTFUL

- La API RESTful (Application Programming Interface REpresentational State Transfer) es una interfaz que dos sistemas de computación utilizan para intercambiar información de manera segura a través de Internet.

- La mayoría de las aplicaciones para empresas deben comunicarse con otras aplicaciones internas o de terceros para llevar a cabo varias tareas.

- Por ejemplo, para generar nóminas mensuales, su sistema interno de cuentas debe compartir datos con el sistema bancario de su cliente para automatizar la facturación y comunicarse con una aplicación interna de planillas de horarios.

- Las API RESTful admiten este intercambio de información porque siguen estándares de comunicación de software seguros, confiables y eficientes.

INTRODUCCIÓN: API

- Una interfaz de programa de aplicación (API) define las reglas que se deben seguir para comunicarse con otros sistemas de software.
- Los desarrolladores exponen o crean API para que otras aplicaciones puedan comunicarse con sus aplicaciones mediante programación.
- Por ejemplo, una aplicación de planificación de horarios puede exponer una API que solicita el nombre completo de un empleado y un rango de fechas.
- Cuando recibe esta información, procesa internamente la planilla de horarios del empleado y devuelve la cantidad de horas trabajadas en ese rango de fechas.
- Se puede pensar en una API web como una puerta de enlace entre los clientes y los recursos de la Web.

FUNCIONAMIENTO DE UNA API RESTFULL

La función básica de una API RESTful es la misma que navegar por Internet. Cuando requiere un recurso, el cliente se pone en contacto con el servidor mediante la API.

Los desarrolladores de API explican cómo el cliente debe utilizar la API REST en la documentación de la API de la aplicación del servidor

A continuación, se indican los pasos generales para cualquier llamada a la API REST:

1. El cliente envía una solicitud al servidor. El cliente sigue la documentación de la API para dar formato a la solicitud de una manera que el servidor comprenda.

2. El servidor autentica al cliente y confirma que este tiene el derecho de hacer dicha solicitud.

3. El servidor recibe la solicitud y la procesa internamente.

4. Luego, devuelve una respuesta al cliente. Esta respuesta contiene información que dice al cliente si la solicitud se procesó de manera correcta. La respuesta también incluye cualquier información que el cliente haya solicitado.

PYTHON FLASK

Flask es un “micro” Framework escrito en Python y desarrollado para simplificar y hacer más fácil la creación de Aplicaciones Web bajo el patrón MVC:

- Micro Framework: Perfecto si se quiere desarrollar una App básica o que se quiera crear de manera rápida y ágil.
- Incluye un servidor web de desarrollo: Por lo tanto, no requiere una infraestructura con un servidor web para probar las apps.
- Tiene un depurador y soporte integrado para pruebas unitarias
- Buen manejo de rutas: Cuando se trabaja con Web Apps hechas en Flask, el controlador que recibe todas las peticiones que hacen los clientes y determina a qué ruta está accediendo el cliente para ejecutar el código pertinente.
- Documentación, lista de correos y código de GitHub.

EMPEZANDO CON FLASK

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World!'

if __name__ == '__main__':
    app.run(debug=True)
```

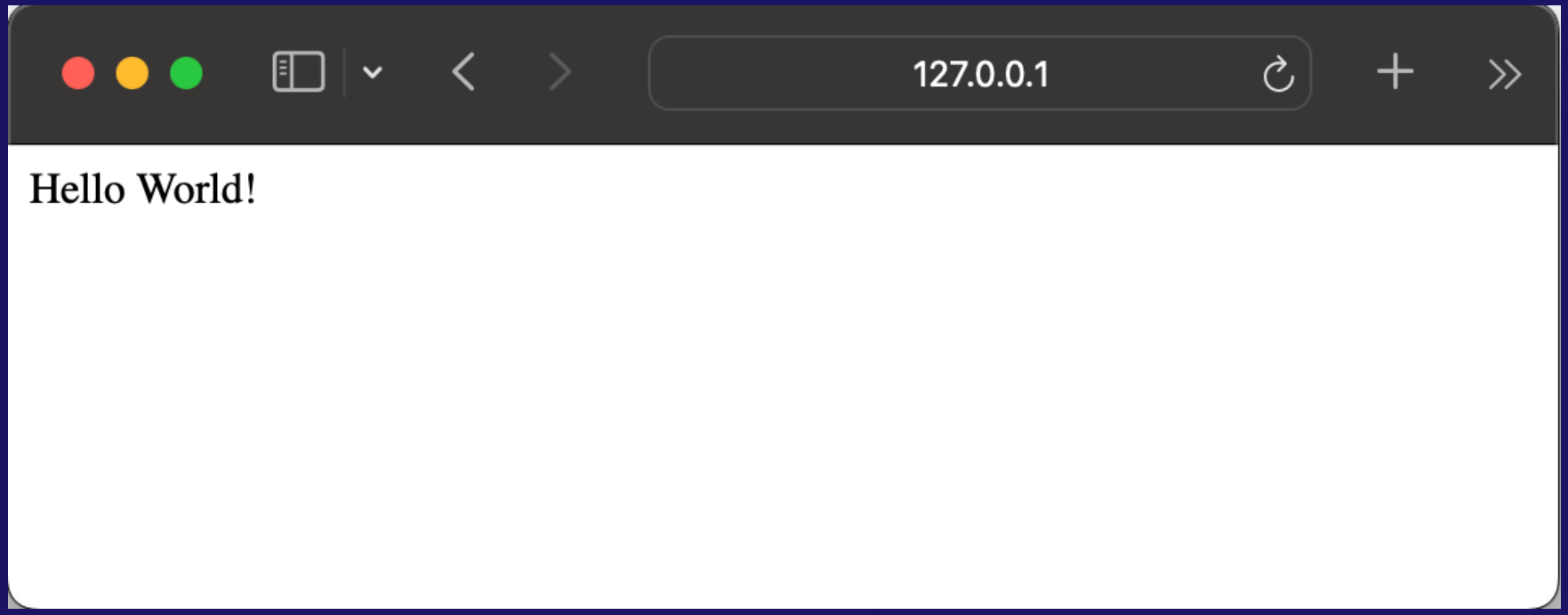
Instala 'flask' usando pip en un venv

EMPEZANDO CON FLASK

```
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not
use it in a production deployment. Use a
production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 111-090-681
```

Ejecutamos

PROBAMOS NUESTRA WEB APP



PROBAMOS NUESTRA WEB APP

```
# tests.py

import requests
r = requests.get('http://127.0.0.1:5000')
print(r)
print(r.status_code)
print(r.text)

# <Response [200]>
# 200
# Hello World!
```

O mejor la probamos desde Python. Para ello debemos ejecutar las pruebas mientras la app de Flask se ejecuta en paralelo

Necesitamos instalar 'requests' con pip

CRUD

CRUD es el acrónimo de Create (Crear), Read (Leer), Update (Actualizar) y Delete (Borrar). Este concepto se utiliza para describir las cuatro operaciones básicas que pueden realizarse en la mayoría de las bases de datos y sistemas de gestión de información.

Estas operaciones permiten a los usuarios crear nuevos datos (Create), leer los existentes (Read), actualizarlos (Update) y eliminarlos (Delete). CRUD es una función esencial para muchos sistemas de información, ya que permite a los usuarios realizar tareas básicas de mantenimiento y gestión de datos.

Vamos a usar Flask para crear un CRUD sobre un diccionario de Python. Todavía no tendrá persistencia (no se guardará en disco).

DATOS Y PERSISTENCIA

```
"""Aqúi almacenaremos los datos
de nuestra app

Estos datos no se guardarán en
disco"""

users = {}
data = {}
```

LISTADO DE DATOS

Ruta

Método

```
@app.route('/data', methods=['GET'])
def get_data():
    return list(data.keys()), 200
```

Código de estado

LISTADO DE DATOS

Método

Ruta

```
r = requests.get('http://127.0.0.1:5000/data')
print(r.status_code)
print(r.text)

# 200
# []
```

Código de estado

Datos (vacío)

MÉTODOS HTTP

Los desarrolladores a menudo implementan API RESTful mediante el uso del protocolo de transferencia de hipertexto (HTTP). Un método de HTTP informa al servidor lo que debe hacer con el recurso. A continuación, se indican cuatro métodos de HTTP comunes:

- **GET** Los clientes utilizan GET para acceder a los recursos que están ubicados en el URL especificado en el servidor.

- **POST** Los clientes usan POST para enviar datos al servidor. Incluyen la representación de los datos con la solicitud. Enviar la misma solicitud POST varias veces produce el efecto secundario de crear el mismo recurso varias veces.

- **PUT** Los clientes utilizan PUT para actualizar los recursos existentes en el servidor. A diferencia de POST, el envío de la misma solicitud PUT varias veces en un servicio web RESTful da el mismo resultado.

- **DELETE** Los clientes utilizan la solicitud DELETE para eliminar el recurso. Una solicitud DELETE puede cambiar el estado del servidor.

CÓDIDOS DE ESTADO

La respuesta a una petición a una API RESTful incluye código de estado de tres dígitos que comunica si la solicitud se procesó de manera correcta o dio error. Por ejemplo, los códigos 2XX indican el procesamiento correcto, pero los códigos 4XX y 5XX indican errores. Los códigos 3XX indican la redirección de URL.

A continuación, se enumeran algunos códigos de estado comunes:

- 200: respuesta genérica de procesamiento correcto
- 401: acceso a recurso no autorizado
- 404: recurso no encontrado
- 409: conflicto con un recurso
- 500: error interno del servidor

La lista completa de códigos puede encontrarse en https://developer.mozilla.org/es/docs/Web/HTTP/Status (recomendación: buscar el código 418 😂 🫖 ☕)

CREANDO DATOS

```
from flask import request

@app.route('/data/<string:id>', methods=['POST'])
def add_data(id):
    if id not in data:
        data[id] = request.args.get('value', '')
        return f'Dato {id} añadido', 200
    else:
        return f'Dato {id} ya existe', 409
```

Variable como parte de URL

Método

Variable como argumento en URL

VARIABLES EN URL

Es posible añadir variables a una URL marcando las variables con <nombre_variable>. La función recibe <nombre_variable> como argumento. Opcionalmente, se puede utilizar un convertidor para especificar el tipo de argumento como <convertidor:nombre_variable>:

- string (por defecto): acepta cualquier texto sin /
- int: acepta números enteros positivos
- float: acepta valores positivos en coma flotante
- path: como string, pero también acepta barras /
- uuid: acepta cadenas UUID

- Para acceder a los parámetros enviados en la URL (?clave=valor) podemos utilizar el atributo args:

```
atributo_a_obtener = request.args.get('key', '')
```

- Se recomienda acceder a los parámetros de la URL con get o capturando el KeyError

CREANDO DATOS

```
URL = 'http://127.0.0.1:5000/'
r = requests.post(f'{URL}/data/dato1?value=valor1')
print(r.status_code)
print(r.text)

# 200
# Dato dato1 añadido
```

ACCEDIENDO A DATOS

```
@app.route('/data/<id>', methods=['GET'])
def get_data_id(id):
    try:
        return data[id], 200
    except KeyError:
        return f'Dato {id} No encontrado', 404
```

ACCEDIENDO A DATOS

```
r = requests.get(f'{URL}/data/dato1')
print(r.status_code)
print(r.text)

# 200
# valor1
```

ACTUALIZANDO DATOS

```
@app.route('/data/<id>', methods=['PUT'])
def update_data(id):
    if id in data:
        data[id] = request.args.get('value', '')
        return f'Dato {id} actualizado', 200
    else:
        return f'Dato {id} No encontrado', 404
```

ACTUALIZANDO DATOS

```
r = requests.put(f'{URL}/data/dato1?value=valorN')
print(r.status_code)
print(r.text)
r = requests.get(f'{URL}/data/dato1')
print(r.text)

# 200
# Dato dato1 actualizado
# valorN
```

ELIMINANDO DATOS

```
@app.route('/data/<id>', methods=['DELETE'])
def delete_data(id):
    if id in data:
        del data[id]
        return f'Dato {id} eliminado', 200
    else:
        return f'Dato {id} No encontrado', 404
```

ELIMINANDO DATOS

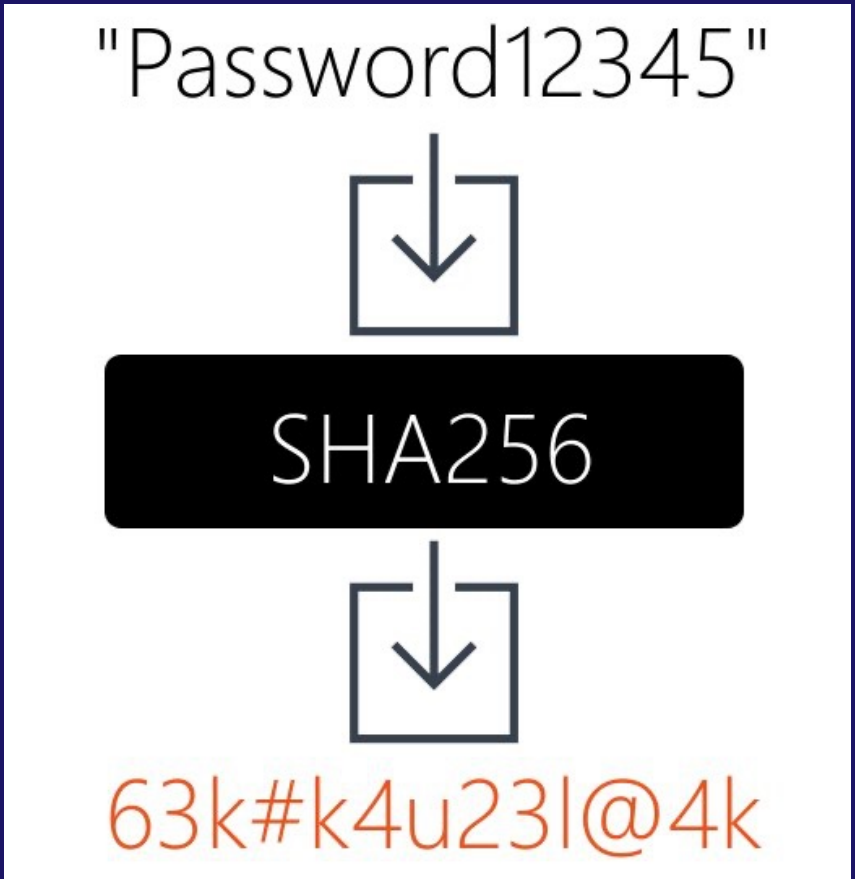
```
r = requests.delete(f'{URL}/data/dato1')
print(r.status_code)
print(r.text)

# 200
# Dato dato1 eliminado
```

AÑADIENDO SEGURIDAD A NUESTRA WEB APP

- Vamos a añadir seguridad a nuestra web app mediante un sistema (básico) de autenticación por usuario / contraseña
- No debemos guardar la contraseña del usuario, sino el HASH de la contraseña
- Además, dado que una API RESTfull es "stateless", usaremos autenticación mediate tokens JWT

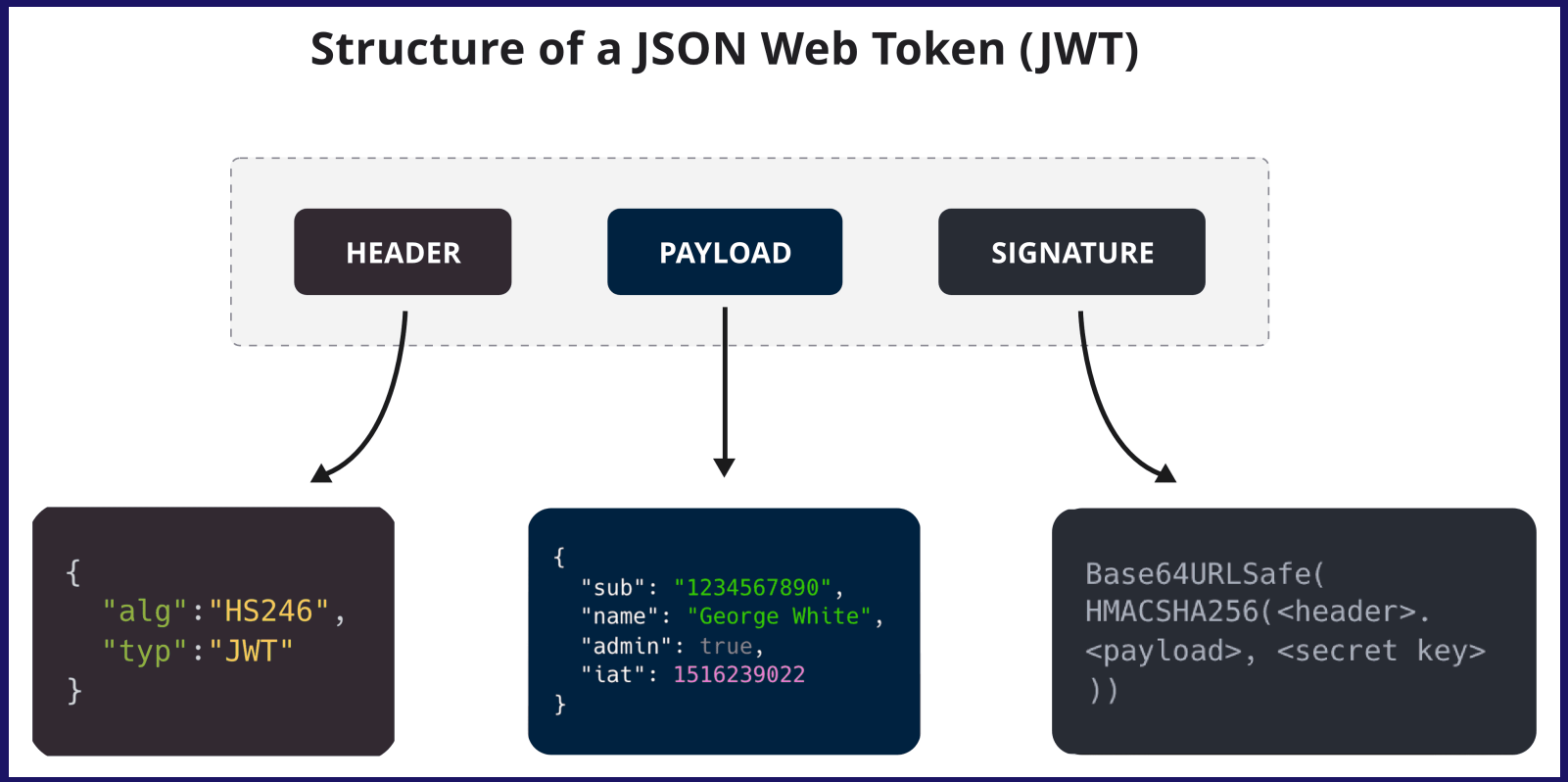
HASHING DE CONTRASEÑAS

SI



NO

JWT


Structure of a JSON Web Token (JWT)
HEADER
PAYLOAD
SIGNATURE
{
"alg":"HS246",
"typ":"JWT"
}
{
"sub": "1234567890",
"name": "George White",
"admin": true,
"iat": 1516239022
}
Base64URLSafe(
HMACSHA256(<header>.
<payload>, <secret key>
))

HABILITADO JWT

```
from flask_jwt_extended import (
    JWTManager, create_access_token, jwt_required,
    get_jwt_identity, get_jwt
)

app = Flask(__name__)

app.config["JWT_SECRET_KEY"] = "super-secret"
jwt = JWTManager(app)
```

¡Cambiadla!

CREAR USUARIOS

```
@app.route('/signup', methods=['POST'])
def signup():
    user = request.args.get('user', '')
    if user in users:
        return f'Usuario {user} ya existe', 409
    else:
        password = request.args.get('password', '')
        hashed =\
           hashlib.sha256(password.encode()).hexdigest()
        users[user] = hashed
        return f'Usuario {user} registrado', 200
```

CREAR USUARIOS

```
r =
requests.post(f'{URL}/signup?user=usuario&password=Pass
w0rd')
print(r.status_code)
print(r.text)

# 200
# Usuario usuario registrado
```

AUTENTICACIÓN

```
@app.route('/signin', methods=['GET'])
def login():
    user = request.args.get('user', '')
    password = request.args.get('password', '')
    hashed = hashlib.sha256(password.encode()).hexdigest()

    if user in users and users[user] == hashed:
        return create_access_token(identity=user), 200
    else:
        return f'Usuario o contraseña incorrectos', 401
```

AUTENTICACIÓN

```
r =
requests.get(f'{URL}/signin?user=usuario&password=Pass
w0rd')
print(r.status_code)
print(r.text)

# 200
#
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc
2UsImlhdCI6MTcxNDQ3NDI5OSwianRpIjoiZjlmNmVhNzgtZGFjZC0
0NjE2LWI0ZGMtZjVjZDk2MTBhMWJlIiwidHlwZSI6ImFjY2VzcyIsI
nN1YiI6InVzdWFyaW8iLCJuYmYiOjE3MTQ0NzQyOTksImNzcmYiOiJ
kYTExMTJkZS03NDAyLTRjNmItODdmMC0xYjc1Y2YyNzQ5ZWYiLCJle
HAiOjE3MTQ0NzUxOTl9.I6U-vw7rqo0oOZBL-
kQnbW53AnrtfqXMVNDmjP4bvmA
```

ANALIZANDO EL TOKEN

JWT

Debugger Libraries Introduction Ask

Crafted by Auth0 by Okta

Encoded PASTE A TOKEN HERE


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JmcmVzaCI6ZmFsc2UsImlhdCI6MTcxNDQ3NDI5O
SwianRpIjoiZjlmNmVhNzgtZGFjZC00NjE2LWI0
ZGMtZjVjZDk2MTBhMWJlIiwidHlwZSI6ImFjY2V
zcyIsInN1YiI6InVzdWFyaW8iLCJuYmYiOjE3MT
Q0NzQyOTksImNzcmYiOiJkYTExMTJkZS03NDAyL
TRjNmItODdmMC0xYjc1Y2YyNzQ5ZWYiLCJleHAi
OjE3MTQ0NzUxOTl9.I6U-vw7rqo0oOZBL-
kQnbW53AnrtfqXMVNDmjP4bvmA
```


Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE


```
{
  "alg": "HS256",
  "typ": "JWT"
}
```


PAYLOAD: DATA


```
{
  "fresh": false,
  "iat": 1714474299,
  "jti": "f9f6ea78-dacd-4616-b4dc-f5cd9610a1be",
  "type": "access",
  "sub": "usuario",
  "nbf": 1714474299,
  "csrf": "da1112de-7402-4c6b-87f0-1b75cf2749ef",
  "exp": 1714475199
}
```


VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  super-secret
) secret base64 encoded
```

Signature Verified

SHARE JWT

AÑADIENDO SEGURIDAD A FUNCIONES

```
@app.route('/data', methods=['GET'])
@jwt_required()
def get_data():
    # Podemos obtener el usuario
    user = get_jwt_identity()
    return list(data.keys()), 200
```

AÑADIENDO SEGURIDAD A FUNCIONES

```
r = requests.get('http://127.0.0.1:5000/data')
print(r.status_code)
print(r.text)

r =
requests.get(f'{URL}/signin?user=usuario&password=Passw0rd')
token = r.text

r = requests.get('http://127.0.0.1:5000/data',\
            headers={'Authorization': 'Bearer ' + token})
print(r.status_code)
print(r.text)

"""
401
{
  "msg": "Missing Authorization Header"
}

200
[
  "dato1"
]
"""
```

TAREAS

- Implementa la API anterior con Flask
- Haz que vuestro trabajo grupal exponga su funcionalidad usando Flask, como hemos visto en los ejemplos anteriores
- Crea un script `examples.py` que implemente una interfaz textual, la cual llame a tu API usando el módulo `requests`
- Añade una descripción de las posibles llamadas a tu API en el fichero `README.md`
- Extra:
  - Usa una API de 3ºs en tu trabajo
  - Haz que tu API tenga distintos tipos de usuarios
  - Implementa la funcionalidad de cerrar sesión (`logout`)
  - Para todo ello, puedes usar como referencia el repositorio de ejemplo